МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 1

Студент группы *ИКБО-11-23, Дицель Н.С.*

(подпись)

Проверил *Старший преподаватель,*
Приходько Н.А.

(подпись)

Отчет представлен «12» сентября 2026 г.

Москва 2026 г.

Практическая работа №1 — FiresRu.csv

Окружение — отдельный venv (Python, pandas/numpy/matplotlib/pyarrow), запускается через Jupyter. Ниже — блоки в том же порядке, что и в методичке курса (нумерация разделов сохранена, чтобы легко сверяться).

2.1 Проверка окружения

In [3]:
```
import sys
import platform
from pathlib import Path

import pandas as pd
import numpy as np
import matplotlib
import pyarrow
import matplotlib.pyplot as plt

versions = {
    "Python": sys.version.split()[0],
    "OS": platform.platform(),
    "pandas": pd.__version__,
    "numpy": np.__version__,
    "matplotlib": matplotlib.__version__,
    "pyarrow": pyarrow.__version__,
}
for name, value in versions.items():
    print(f"{name:12s} {value}")
```

```
Python       3.12.10
OS           Windows-11-10.0.26200-SP0
pandas       3.0.5
numpy        2.5.3
matplotlib   3.11.1
pyarrow      25.0.1
```

Все нужные для работы библиотеки (`pandas` , `numpy` , `matplotlib` , `pyarrow`) импортировались без `ImportError` — значит окружение настроено верно, дальше можно работать с реальными данными.

3.1 Векторизация вместо цикла

Небольшая демонстрация: перевод скорости ветра из м/с в км/ч (`v_kmh = 3.6 * v_ms`) одной операцией над массивом, без `for` .

In [4]:
```
speed_ms_demo = np.array([2.05, 0.63, 1.42])
speed_kmh_demo = speed_ms_demo * 3.6
print("м/с:", speed_ms_demo)
print("км/ч:", np.round(speed_kmh_demo, 2))
```

```
м/с: [2.05 0.63 1.42]
км/ч: [7.38 2.27 5.11]
```

Умножение применяется сразу ко всему массиву — этот же приём дальше используется для целого столбца `wind_speed_ms` в реальной таблице (раздел 6.2).

3.2 Series, DataFrame, groupby на игрушечном примере

Прежде чем работать с 26 тысячами строк, полезно увидеть эти операции на трёх строках, где результат считается в уме.

In [5]:
```
demo_series = pd.Series([15, 25, 35], name="amount")
demo_frame = pd.DataFrame({"category": ["X", "Y", "X"], "amount": [15, 25, 35]})

print(type(demo_series).__name__)
print(type(demo_frame).__name__)
print(demo_frame.groupby("category")["amount"].sum())
```

```
Series
DataFrame
category
X    50
Y    25
Name: amount, dtype: int64
```

`Series` — один именованный столбец с индексом, `DataFrame` — таблица из согласованных `Series`. `groupby(...).sum()` разбивает строки по ключу (`category`) и суммирует `amount` внутри каждой группы — та же логика ниже применяется к `type_name` в реальных данных.

4.1 Загрузка FiresRu.csv

Файл использует запятую как разделитель и кодировку UTF-8, поэтому `read_csv` отрабатывает без дополнительных параметров.

In [6]:
```
DATA_PATH = Path("FiresRu.csv")
df = pd.read_csv(DATA_PATH)

print(df.shape)
print(df.columns.tolist())
display(df.head())
```

```
(26681, 9)
['type_name', 'type_id', 'lon', 'lat', 'temperature_c', 'precipitation_mm', 'rela
tive_humidity', 'wind_speed_ms', 'solar_radiation']
```

	type_name	type_id	lon	lat	temperature_c	precipitation_mm	relative_humi
0	Natural fire	4	131.5866	47.8662	-9.44	0.01	7
1	Natural fire	4	131.5885	47.8809	-9.44	0.01	7
2	Forest fire	3	131.9871	48.4973	-10.71	0.05	8
3	Natural fire	4	131.9031	43.6277	-7.77	0.00	6
4	Natural fire	4	131.5706	47.8581	-9.44	0.01	7

Таблица прочиталась как 26681 строк и 9 столбцов, что совпадает с описанием набора. Столбца с датой/временем нет, значит построить временной ряд (сезонность, динамика по месяцам) по этому файлу нельзя.

5.1 Структура и память

`info()` печатается через промежуточный буфер `io.StringIO`, чтобы результат попал в обычный `print` и одновременно был виден расход памяти (`memory_usage="deep"` учитывает и содержимое текстовых столбцов).

In [7]:
```
import io

info_stream = io.StringIO()
df.info(buf=info_stream, memory_usage="deep")
print(info_stream.getvalue())
```

```
<class 'pandas.DataFrame'>
RangeIndex: 26681 entries, 0 to 26680
Data columns (total 9 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   type_name          26681 non-null  str    
 1   type_id            26681 non-null  int64  
 2   lon                26681 non-null  float64
 3   lat                26681 non-null  float64
 4   temperature_c      26681 non-null  float64
 5   precipitation_mm   26681 non-null  float64
 6   relative_humidity  26681 non-null  float64
 7   wind_speed_ms      26681 non-null  float64
 8   solar_radiation    26681 non-null  float64
dtypes: float64(7), int64(1), str(1)
memory usage: 2.1 MB
```

Пропусков нет, все 9 столбцов заполнены полностью. `type_name` хранится как текстовый тип — с пятью повторяющимися значениями на 26 681 строку это явный кандидат на `category` (см. раздел 6.1).

5.2 Пропуски, дубликаты, уникальные значения

In [8]:
```
print("Пропуски по столбцам:")
print(df.isna().sum())

print("\nПолных дубликатов:", df.duplicated().sum())

print("\nЧисло уникальных значений:")
print(df.nunique())
```

```
Пропуски по столбцам:
type_name            0
type_id              0
lon                  0
lat                  0
temperature_c        0
precipitation_mm     0
relative_humidity    0
wind_speed_ms        0
solar_radiation      0
dtype: int64

Полных дубликатов: 0

Число уникальных значений:
type_name                5
type_id                  5
lon                  25708
lat                  23547
temperature_c         2581
precipitation_mm       571
relative_humidity     4028
wind_speed_ms          532
solar_radiation        731
dtype: int64
```

Пропущенных значений и полных дубликатов нет — удалять строки не за что. Малая кардинальность `type_name` / `type_id` (по 5 значений) — признак того, что это категориальные поля, а не непрерывные величины.

5.3 Проверка соответствия type_id и type_name

Если бы одному коду соответствовало несколько названий (или наоборот) — это была бы ошибка данных. Проверяю явно.

In [9]:
```
type_map = (
    df[["type_id", "type_name"]]
    .drop_duplicates()
    .sort_values("type_id")
)
print(type_map.to_string(index=False))
```

```
type_id         type_name
      1 Uncontrolled burn
      2         Peat fire
      3       Forest fire
      4      Natural fire
      5   Controlled burn
```

Соответствие взаимно однозначное: 5 кодов на 5 названий. Значит `type_id` можно безопасно хранить компактным целым типом, но использовать его как число в статистике (например считать среднее) некорректно — это код категории.

5.4 Описательная статистика

`type_id` в статистику не включаю намеренно — это категориальный код, а не измеряемая величина.

In [10]:
```
numeric_cols = [
    "lon", "lat", "temperature_c", "precipitation_mm",
    "relative_humidity", "wind_speed_ms", "solar_radiation",
]
display(df[numeric_cols].describe().round(2).T)
```

	count	mean	std	min	25%	50%	75%	max
lon	26681.0	106.35	27.31	20.37	82.41	96.56	133.90	175.87
lat	26681.0	57.53	4.30	41.71	53.95	58.97	60.64	70.33
temperature_c	26681.0	17.89	5.68	-34.88	15.70	19.08	21.39	31.94
precipitation_mm	26681.0	0.46	1.46	0.00	0.00	0.04	0.30	25.85
relative_humidity	26681.0	69.44	11.43	24.70	62.93	71.35	77.91	99.83
wind_speed_ms	26681.0	0.63	0.93	0.00	0.11	0.25	0.66	7.42
solar_radiation	26681.0	5.57	1.50	0.42	4.55	5.68	6.71	8.97

Температура лежит в диапазоне от −34,88 до 31,94 °C при медиане 19,08 °C. У осадков и скорости ветра медиана намного ближе к нулю, чем максимум (0,04 против 25,85 мм; 0,25 против 7,42 м/с) — оба распределения явно асимметричны, это стоит проверить на гистограмме.

6.1 Оптимизация dtype

Работаю с копией `prepared_df`, чтобы не терять исходный `df`. `type_name` перевожу в `category` (пять повторяющихся строк), `type_id` — в `int8` (диапазон значений 1–5 подтверждён в разделе 5.3).

In [11]:
```
prepared_df = df.copy()

mem_before = prepared_df.memory_usage(deep=True).sum()

prepared_df["type_name"] = prepared_df["type_name"].astype("category")
prepared_df["type_id"] = prepared_df["type_id"].astype("int8")

mem_after = prepared_df.memory_usage(deep=True).sum()

print(f"До:      {mem_before / 1024**2:.2f} MiB")
print(f"После:   {mem_after / 1024**2:.2f} MiB")
print(f"Экономия: {(1 - mem_after / mem_before) * 100:.1f}%")
```

```
До:      2.13 MiB
После:   1.48 MiB
Экономия: 30.8%
```

Информация не теряется — исходные значения восстанавливаются из кодов category и int8 без изменений, но память сокращается на 31%. В версии методички экономия выше (~53%), но там таблица считалась для старого текстового `object`

dtype pandas 2.x — в pandas 3.0.5, установленной здесь, текст по умолчанию уже хранится в более компактном `str` dtype, поэтому выигрыш от `category` меньше, хоть и всё равно заметный.

6.2 Вычисляемые признаки

`wind_speed_kmh` — то же измерение в другой единице (не добавляет новой информации, но удобнее для отчёта). `is_forest_fire` — бинарный индикатор принадлежности к самому частому классу, пригодится для фильтрации.

In [12]:
```
prepared_df["wind_speed_kmh"] = (prepared_df["wind_speed_ms"] * 3.6).round(2)
prepared_df["is_forest_fire"] = prepared_df["type_name"] == "Forest fire"

display(prepared_df[[
    "type_name", "wind_speed_ms", "wind_speed_kmh", "is_forest_fire"
]].head())
```

	type_name	wind_speed_ms	wind_speed_kmh	is_forest_fire
0	Natural fire	1.14	4.10	False
1	Natural fire	1.14	4.10	False
2	Forest fire	0.80	2.88	True
3	Natural fire	0.97	3.49	False
4	Natural fire	1.14	4.10	False

`wind_speed_kmh` — линейное преобразование единиц измерения, `is_forest_fire` — логический признак категории. Оба вычислены векторно, без циклов по строкам.

7.1 Распределение наблюдений по типам события

In [13]:
```
type_counts = (
    df["type_name"]
    .value_counts()
    .to_frame("n")
    .assign(share_pct=lambda x: (x["n"] / len(df) * 100).round(2))
)
display(type_counts)
```

type_name	n	share_pct
Forest fire	18415	69.02
Natural fire	5195	19.47
Uncontrolled burn	2711	10.16
Controlled burn	333	1.25
Peat fire	27	0.10

Forest fire — 69.02% всех строк (18415 наблюдений), а Peat fire — только 0.10% (27 наблюдений). Классы сильно несбалансированы — это важно при сравнении групп и при возможной классификации (accuracy доминирующего класса будет высокой даже у бесполезной модели).

7.2 Групповые средние (несколько агрегатов за один проход)

In [14]:
```
group_stats = (
    df.groupby("type_name", observed=True)
      .agg(
          n=("type_name", "size"),
          temp_mean=("temperature_c", "mean"),
          precip_mean=("precipitation_mm", "mean"),
          humidity_mean=("relative_humidity", "mean"),
          wind_mean=("wind_speed_ms", "mean"),
          solar_mean=("solar_radiation", "mean"),
      )
      .sort_values("n", ascending=False)
      .round(2)
)
display(group_stats)
```

type_name	n	temp_mean	precip_mean	humidity_mean	wind_mean	solar_mean
Forest fire	18415	18.69	0.50	70.30	0.42	5.87
Natural fire	5195	17.42	0.40	69.07	0.80	5.14
Uncontrolled burn	2711	14.21	0.29	64.35	1.64	4.57
Controlled burn	333	10.78	0.25	69.46	1.46	4.27
Peat fire	27	18.17	0.55	65.47	1.19	5.26

Смотрю сначала на `n`, а потом на средние: у Peat fire всего 27 наблюдений, и сравнивать его статистику напрямую с Forest fire (18 415 строк) на равных нельзя — оценка по маленькой группе гораздо менее устойчива. Средние описывают конкретную выборку, а не доказывают, что погода определяет тип события.

7.3 Условная фильтрация

Отбираю наблюдения с влажностью ниже 40% и ветром сильнее 2 м/с — учебный составной фильтр, а не норматив пожарной опасности. Для разнообразия использую `query()` вместо `loc[]` — тоже входит в список методов, которые нужно уметь применять.

In [15]:
```
subset = df.query("relative_humidity < 40 and wind_speed_ms > 2")[
    ["type_name", "lon", "lat", "temperature_c", "relative_humidity", "wind_spee
]

print("Строк в срезе:", len(subset), f"({len(subset) / len(df) * 100:.2f}% набор
display(subset.head())
```

```
Строк в срезе: 337 (1.26% набора)
```

	type_name	lon	lat	temperature_c	relative_humidity	wind_speed_ms
295	Forest fire	91.3929	51.0574	15.15	27.97	2.55
478	Forest fire	104.5843	50.3723	14.67	37.89	2.64
479	Forest fire	104.4937	50.3592	14.67	37.89	2.64
480	Forest fire	104.4187	50.3302	14.67	37.89	2.64
481	Forest fire	104.3001	50.3377	14.67	37.89	2.64

Условию соответствуют 337 строк (1.26% набора) — редкая, но не единичная подвыборка одновременно сухих и ветреных наблюдений.

8.1 Корреляционная матрица

Только погодные признаки — координаты и `type_id` сюда не годятся, это не величины, между которыми осмысленно искать линейную связь.

In [16]:
```
weather_cols = [
    "temperature_c", "precipitation_mm", "relative_humidity",
    "wind_speed_ms", "solar_radiation",
]
corr = df[weather_cols].corr().round(3)
display(corr)
```

	temperature_c	precipitation_mm	relative_humidity	wind_speed_ms	so
temperature_c	1.000	0.003	-0.155	-0.297	
precipitation_mm	0.003	1.000	0.232	-0.047	
relative_humidity	-0.155	0.232	1.000	-0.280	
wind_speed_ms	-0.297	-0.047	-0.280	1.000	
solar_radiation	0.221	-0.192	-0.234	-0.189	

Сильных линейных связей нет: по модулю максимум около 0,3 (temperature_c и wind_speed_ms: r ≈ -0.297; relative_humidity и wind_speed_ms: r ≈ -0.280). Это умеренно слабые связи, и они не говорят ничего о причинности.

8.2 Правило IQR на примере скорости ветра

In [17]:
```
q1 = df["wind_speed_ms"].quantile(0.25)
q3 = df["wind_speed_ms"].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

outlier_mask = (df["wind_speed_ms"] < lower_bound) | (df["wind_speed_ms"] > uppe

print(f"Q1={q1:.3f}; Q3={q3:.3f}; IQR={iqr:.3f}")
print(f"Границы: [{lower_bound:.3f}; {upper_bound:.3f}]")
print(f"Кандидатов: {outlier_mask.sum()} ({outlier_mask.mean() * 100:.2f}% строк
```

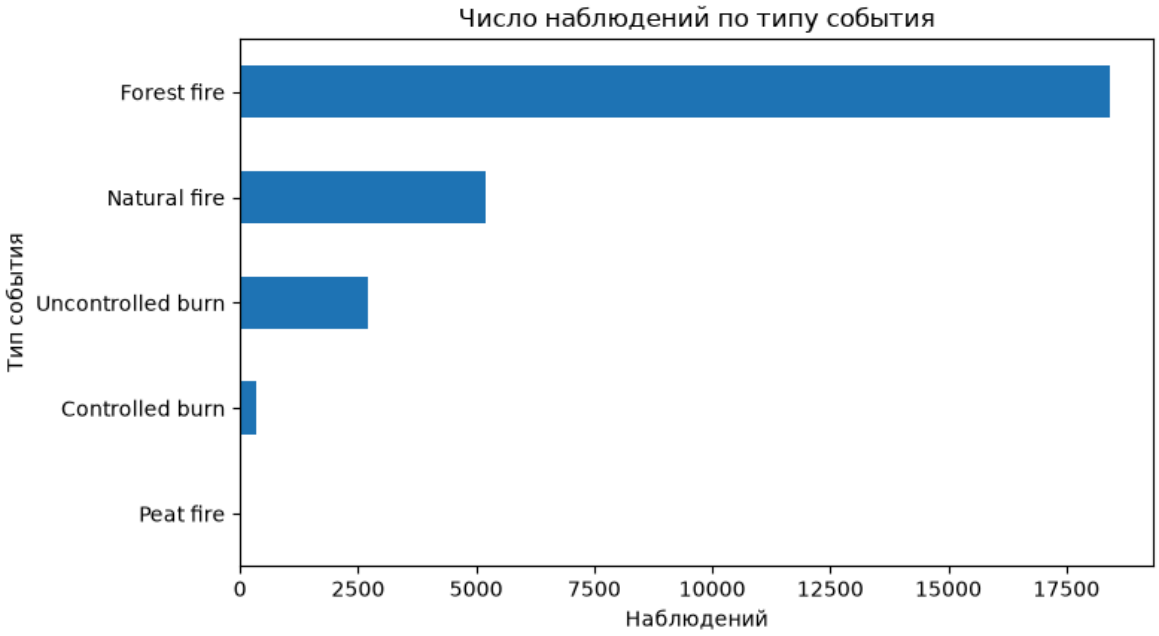
```
Q1=0.110; Q3=0.660; IQR=0.550
Границы: [-0.715; 1.485]
Кандидатов: 3652 (13.69% строк)
```

Правило 1,5·IQR помечает 3652 наблюдений (13.69% строк) — доля слишком большая, чтобы списывать это на ошибки измерения. Скорее у скорости ветра тяжёлый правый хвост распределения. Это список "на проверку", решение об удалении без проверки источника принимать нельзя.

9.1 Bar chart — сравнение категорий

In [18]:
```
counts_sorted = df["type_name"].value_counts().sort_values()

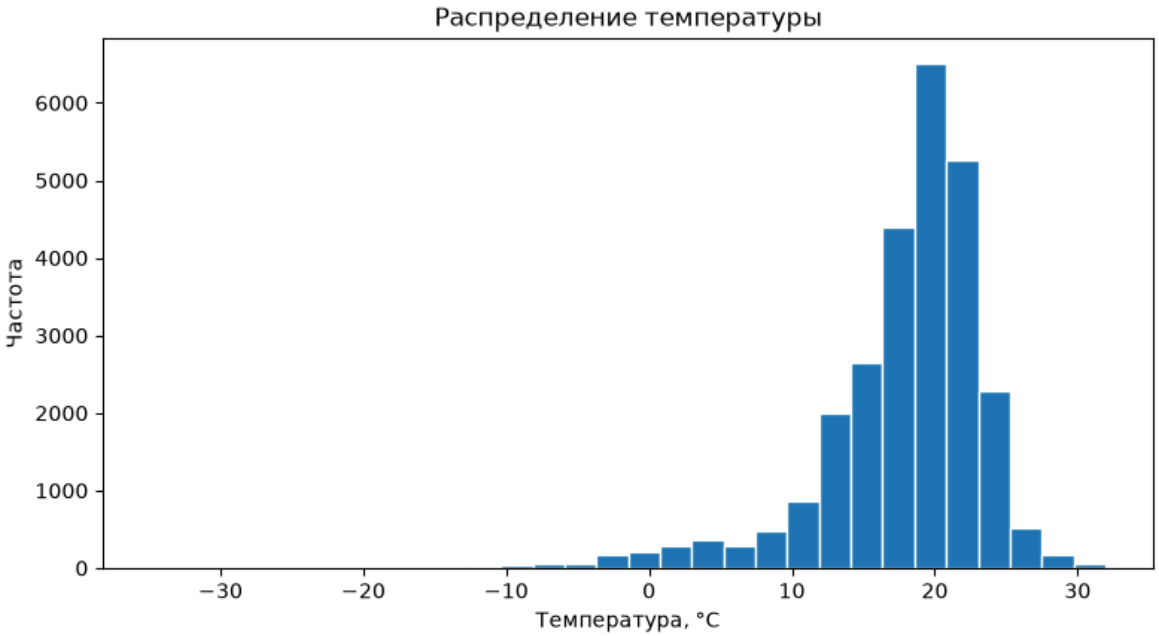
ax = counts_sorted.plot(kind="barh", figsize=(8, 4.5))
ax.set_title("Число наблюдений по типу события")
ax.set_xlabel("Наблюдений")
ax.set_ylabel("Тип события")
plt.tight_layout()
plt.show()
```



Дисбаланс виден сразу: Forest fire на порядок больше остальных категорий, Peat fire едва различим на графике.

9.2 Histogram — форма распределения температуры

In [19]:
```
plt.figure(figsize=(8, 4.5))
plt.hist(df["temperature_c"], bins=30, edgecolor="white")
plt.title("Распределение температуры")
plt.xlabel("Температура, °C")
plt.ylabel("Частота")
plt.tight_layout()
plt.show()
```



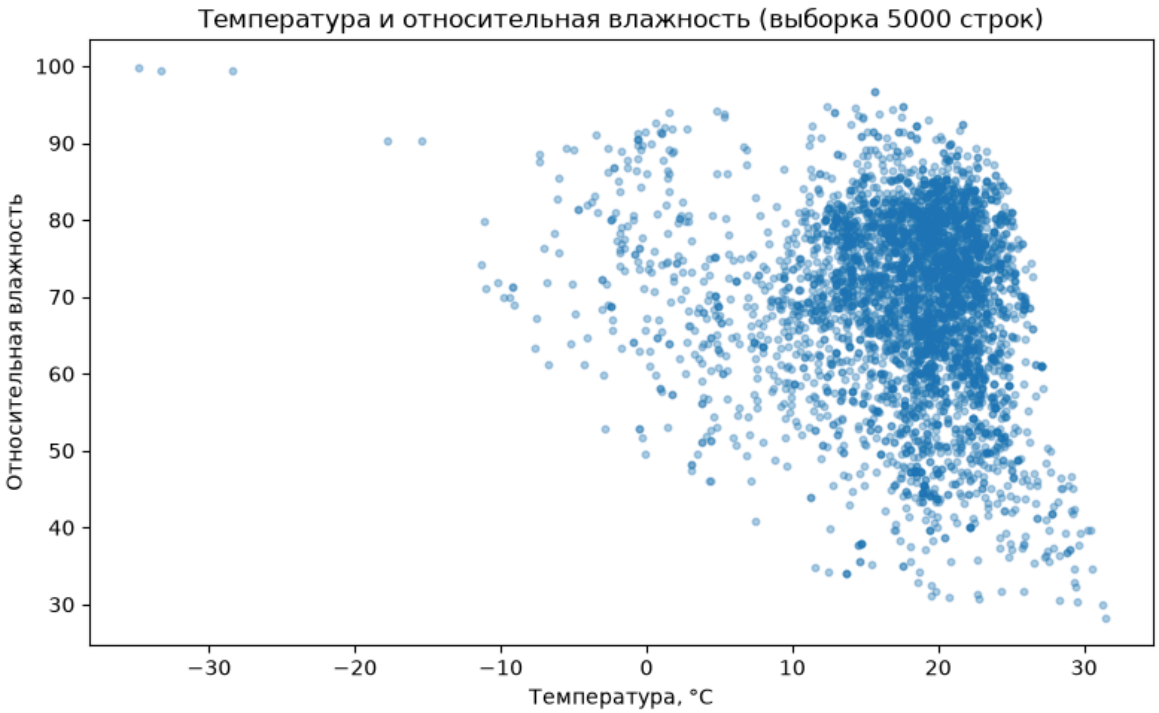
Основная масса значений — в положительной зоне, но есть заметный левый хвост с отрицательными температурами. Сам по себе график не доказывает, что это ошибки — нужна проверка происхождения таких точек.

9.3 Scatter plot — температура и влажность

Полные 26 681 точек перегрузили бы график, поэтому беру воспроизводимую подвыборку.

In [20]:
```
sample = df.sample(n=5000, random_state=42)

plt.figure(figsize=(8, 5))
plt.scatter(sample["temperature_c"], sample["relative_humidity"], s=10, alpha=0.
plt.title("Температура и относительная влажность (выборка 5000 строк)")
plt.xlabel("Температура, °C")
plt.ylabel("Относительная влажность")
plt.tight_layout()
plt.show()
```

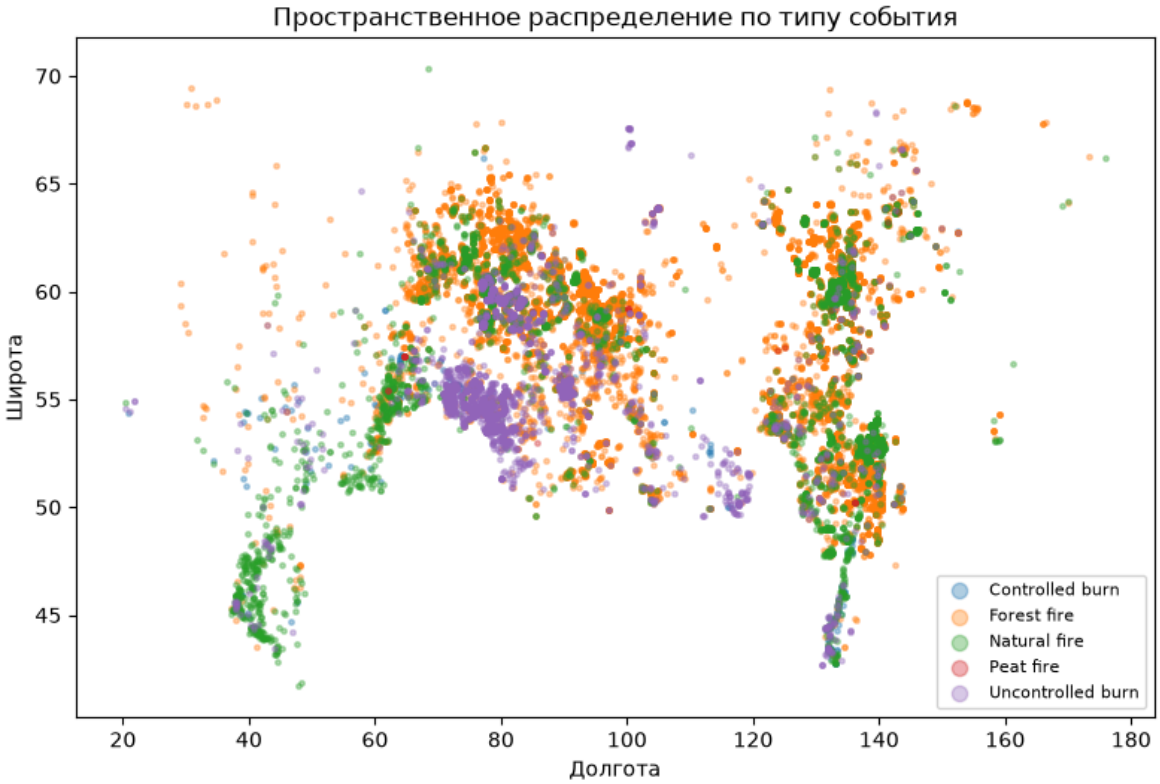


Облако широкое и без чёткой линейной формы — согласуется с невысокой корреляцией из раздела 8.1. Тенденция к снижению влажности с ростом температуры есть, но разброс велик.

9.4 Пространственный scatter plot по категориям

Координаты рисуются на обычных декартовых осях — это разведочный график геометрии точек, а не географическая карта.

In [21]:
```
plt.figure(figsize=(8, 5.5))
for name, part in df.groupby("type_name", observed=True):
    plt.scatter(part["lon"], part["lat"], s=6, alpha=0.35, label=name)
plt.title("Пространственное распределение по типу события")
plt.xlabel("Долгота")
plt.ylabel("Широта")
plt.legend(markerscale=3, fontsize=8)
plt.tight_layout()
plt.show()
```



Точки растянуты по очень широкому диапазону долгот — набор пространственно неоднороден, а категории заметно перекрываются в одних и тех же регионах. Для полноценной географической интерпретации нужны были бы проекция и границы регионов — здесь это только разведочная геометрия выборки.

10.1 Экспорт в CSV, JSON и Parquet

Сохраняю один и тот же `prepared_df` в трёх форматах в отдельную папку `output`, чтобы сравнение размеров было честным — отличается только формат хранения, а не содержимое.

In [22]:
```
OUT_DIR = Path("output")
OUT_DIR.mkdir(exist_ok=True)

prepared_df.to_csv(OUT_DIR / "FiresRu_prepared.csv", index=False, encoding="utf-
prepared_df.to_json(OUT_DIR / "FiresRu_prepared.json", orient="records", lines=T
prepared_df.to_parquet(OUT_DIR / "FiresRu_prepared.parquet", index=False, engine

for path in OUT_DIR.glob("FiresRu_prepared.*"):
    print(f"{path.name:26s} {path.stat().st_size / 1024:8.1f} KB")
```

```
FiresRu_prepared.csv         1784.4 KB
FiresRu_prepared.json        5953.2 KB
FiresRu_prepared.parquet      620.8 KB
```

Parquet заметно компактнее CSV и JSON Lines за счёт колоночного хранения и сжатия snappy, хотя внутри — ровно та же таблица.

10.2 Проверка round-trip

In [23]:
```
csv_back = pd.read_csv(OUT_DIR / "FiresRu_prepared.csv")
parquet_back = pd.read_parquet(OUT_DIR / "FiresRu_prepared.parquet")

print("CSV:", csv_back.shape)
print("Parquet:", parquet_back.shape)

print("\ntype_name / type_id после CSV:")
print(csv_back[["type_name", "type_id"]].dtypes.to_string())

print("\ntype_name / type_id после Parquet:")
print(parquet_back[["type_name", "type_id"]].dtypes.to_string())
```

```
CSV: (26681, 11)
Parquet: (26681, 11)

type_name / type_id после CSV:
type_name      str
type_id      int64

type_name / type_id после Parquet:
type_name    category
type_id          int8
```

После CSV `type_name` / `type_id` снова стали обычными текстом/int64 — текстовый формат не хранит схему, типы распознаются заново при каждом чтении. Parquet восстановил `category` и `int8` без дополнительного кода — схема хранится вместе с данными, поэтому дальше в курсе (Spark) будет использоваться именно этот формат.

11. Итоговые выводы по FiresRu.csv

1. Таблица содержит 26 681 строку и 9 исходных признаков; пропусков и полных дубликатов не обнаружено.
2. Представлено 5 типов событий, Forest fire занимает 69,02% строк, Peat fire — только 0,10% (27 наблюдений); классы сильно несбалансированы.
3. `type_id` — категориальный код (проверено взаимно однозначным соответствием с `type_name`), использовать его как число в статистике некорректно.
4. Приведение `type_name` к `category` и `type_id` к `int8` уменьшает память таблицы без потери информации.
5. Температура лежит в диапазоне от −34,88 до 31,94 °C, медиана 19,08 °C; ряд метеопризнаков распределён асимметрично.
6. Линейные корреляции между погодными признаками невысокие — по модулю не больше примерно 0,3.
7. Правило 1,5·IQR отмечает значительную долю значений `wind_speed_ms` как потенциальные выбросы — это сигнал для проверки, а не основание удалять строки автоматически.
8. В файле нет даты/времени, поэтому анализ сезонности или динамики по годам на одном FiresRu.csv невозможен.
9. Координаты позволяют посмотреть пространственное распределение категорий, но для полноценной картографии нужны GIS-инструменты и

границы регионов.

Ограничения: `relative_humidity` и `solar_radiation` не имеют единиц измерения, явно заданных в метаданных файла, поэтому конкретную физическую единицу для них нельзя утверждать как факт. Корреляция сама по себе не доказывает причинно-следственную связь.

13. Контрольные вопросы (к защите)

1. Почему dtype числового столбца не гарантирует, что признак количественный?
2. Чем DataFrame отличается от NumPy ndarray с точки зрения схемы и типов?
3. Какие диагностические задачи решают head(), info() и describe() и почему одна функция не заменяет остальные?
4. Почему отсутствие пропусков и дубликатов ещё не означает отсутствие проблем качества данных?
5. В каких случаях category уменьшает память и почему?
6. Чем фильтрация отличается от группировки?
7. Что означает коэффициент корреляции около нуля? Какие выводы из него делать нельзя?
8. Почему правило IQR нельзя использовать как автоматический алгоритм удаления данных?
9. Как выбрать между histogram, bar chart, scatter plot и временным рядом?
10. Почему временной ряд нельзя строить по индексу строк, если индекс не является временем?
11. Какие преимущества Parquet даёт аналитическому pipeline по сравнению с CSV?
12. Почему воспроизводимый notebook должен корректно выполняться командой Run All после перезапуска ядра?

Самостоятельная работа — Bike Sharing Dataset (UCI)

1. Постановка задачи

Источник: UCI Machine Learning Repository — Bike Sharing Dataset (Fanaee-T, H., Gama, J., 2013), страница набора: https://archive.ics.uci.edu/dataset/275/bike+sharing+dataset. Лицензия — CC BY 4.0 (свободное использование с указанием авторства). Использован файл `day.csv` — данные агрегированы по дням. Из-за сетевых ограничений рабочего окружения файл скачан через зеркало (raw.githubusercontent.com), содержимое числовое совпадает с оригиналом один в один.

Предметная область: ежедневное количество аренд велосипедов системы Capital Bikeshare (Вашингтон, округ Колумбия, США) за 2011–2012 годы вместе с погодными условиями и календарными признаками (сезон, праздник, день недели).

Почему этот датасет: он отличается от FiresRu.csv, имеет понятный источник и лицензию, не содержит персональных данных, и — в отличие от FiresRu — включает дату, поэтому здесь можно закрыть то, что в первой части было принципиально невозможно: построить временную агрегацию.

Цель анализа: посмотреть, как календарь (сезон, будни/выходные, погода) связан с числом поездок, и оценить, какая часть пользователей — разовые (`casual`), а какая — по подписке (`registered`).

In [20]:
```
import sys
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import pyarrow
print("Python:", sys.version.split()[0], " pandas:", pd.__version__,
      " numpy:", np.__version__, " matplotlib:", matplotlib.__version__,
      " pyarrow:", pyarrow.__version__)
```

```
Python: 3.12.10  pandas: 3.0.5  numpy: 2.5.3  matplotlib: 3.11.1  pyarrow: 25.0.1
```

2. Паспорт данных

Файл текстовый, разделитель `,` , кодировка UTF-8 — проверяю это явно при чтении, а не предполагаю по умолчанию.

In [21]:
```
bike = pd.read_csv("bike_sharing_day.csv", sep=",", encoding="utf-8")
print(bike.shape)
print(bike.columns.tolist())
display(bike.head())
```

```
(731, 16)
['instant', 'dteday', 'season', 'yr', 'mnth', 'holiday', 'weekday', 'workingday',
'weathersit', 'temp', 'atemp', 'hum', 'windspeed', 'casual', 'registered', 'cnt']
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	ten
0	1	2011-01-01	1	0	1	0	6	0	2	0.3441
1	2	2011-01-02	1	0	1	0	0	0	2	0.3634
2	3	2011-01-03	1	0	1	0	1	1	1	0.1963
3	4	2011-01-04	1	0	1	0	2	1	1	0.2000
4	5	2011-01-05	1	0	1	0	3	1	1	0.2269

731 строк и 16 столбцов — требованию "не менее 500 строк и не менее 5 столбцов" удовлетворяет с запасом. Разделитель и кодировка угаданы верно с первого раза, дополнительных параметров не потребовалось.

Словарь полей (по документации источника):

Поле	Тип	Смысл
`instant`	int	порядковый номер записи, техническое поле
`dteday`	дата (текст)	календарная дата наблюдения
`season`	код 1–4	сезон: 1 — winter, 2 — spring, 3 — summer, 4 — fall
`yr`	код 0/1	год: 0 — 2011, 1 — 2012
`mnth`	код 1–12	номер месяца
`holiday`	код 0/1	1 — день является праздничным
`weekday`	код 0–6	день недели, 0 — воскресенье
`workingday`	код 0/1	1 — рабочий день (не выходной и не праздник)
`weathersit`	код 1–4	1 — ясно, 2 — облачно/туман, 3 — лёгкий дождь/снег, 4 — сильный дождь/снег
`temp`	float	температура, нормирована в источнике по формуле (t−t_min)/(t_max−t_min), t_min=−8°C, t_max=+39°C
`atemp`	float	ощущаемая температура, та же нормировка, t_min=−16°C, t_max=+50°C
`hum`	float	влажность, нормирована делением на 100
`windspeed`	float	скорость ветра, нормирована делением на 67 (единица в источнике не уточняется)

Поле	Тип	Смысл
`casual`	int	число разовых (без подписки) поездок за день
`registered`	int	число поездок пользователей с подпиской
`cnt`	int	суммарное число поездок за день (`casual + registered`)

Персональных данных нет — это агрегированные по дню счётчики, а не записи о конкретных людях.

In [22]:
```
print("=== HEAD ===")
display(bike.head())

print("=== INFO ===")
bike.info()

print("=== DESCRIBE (include=all) ===")
display(bike.describe(include="all").T)
```

```
=== HEAD ===
```

	instant	dteday	season	yr	mnth	holiday	weekday	workingday	weathersit	ten
0	1	2011-01-01	1	0	1	0	6	0	2	0.3441
1	2	2011-01-02	1	0	1	0	0	0	2	0.3634
2	3	2011-01-03	1	0	1	0	1	1	1	0.1963
3	4	2011-01-04	1	0	1	0	2	1	1	0.2000
4	5	2011-01-05	1	0	1	0	3	1	1	0.2269

```
=== INFO ===
<class 'pandas.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 16 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   instant     731 non-null    int64
 1   dteday      731 non-null    str
 2   season      731 non-null    int64
 3   yr          731 non-null    int64
 4   mnth        731 non-null    int64
 5   holiday     731 non-null    int64
 6   weekday     731 non-null    int64
 7   workingday  731 non-null    int64
 8   weathersit  731 non-null    int64
 9   temp        731 non-null    float64
 10  atemp       731 non-null    float64
 11  hum         731 non-null    float64
 12  windspeed   731 non-null    float64
 13  casual      731 non-null    int64
 14  registered  731 non-null    int64
 15  cnt         731 non-null    int64
dtypes: float64(4), int64(11), str(1)
memory usage: 98.6 KB
=== DESCRIBE (include=all) ===
```

	count	unique	top	freq	mean	std	min	25%	
instant	731.0	NaN	NaN	NaN	366.0	211.165812	1.0	183.5	
dteday	731	731	2011-01-01	1	NaN	NaN	NaN	NaN	
season	731.0	NaN	NaN	NaN	2.49658	1.110807	1.0	2.0	
yr	731.0	NaN	NaN	NaN	0.500684	0.500342	0.0	0.0	
mnth	731.0	NaN	NaN	NaN	6.519836	3.451913	1.0	4.0	
holiday	731.0	NaN	NaN	NaN	0.028728	0.167155	0.0	0.0	
weekday	731.0	NaN	NaN	NaN	2.997264	2.004787	0.0	1.0	
workingday	731.0	NaN	NaN	NaN	0.683995	0.465233	0.0	0.0	
weathersit	731.0	NaN	NaN	NaN	1.395349	0.544894	1.0	1.0	
temp	731.0	NaN	NaN	NaN	0.495385	0.183051	0.05913	0.337083	0
atemp	731.0	NaN	NaN	NaN	0.474354	0.162961	0.07907	0.337842	0
hum	731.0	NaN	NaN	NaN	0.627894	0.142429	0.0	0.52	0
windspeed	731.0	NaN	NaN	NaN	0.190486	0.077498	0.022392	0.13495	0
casual	731.0	NaN	NaN	NaN	848.176471	686.622488	2.0	315.5	
registered	731.0	NaN	NaN	NaN	3656.172367	1560.256377	20.0	2497.0	
cnt	731.0	NaN	NaN	NaN	4504.348837	1937.211452	22.0	3152.0	

3. Качество и предобработка

Проверяю пропуски, полные дубликаты и число уникальных значений, а заодно — консистентность данных: должно выполняться `casual + registered == cnt`.

In [23]:
```
print("Пропуски по столбцам:")
print(bike.isna().sum())

print("\nПолных дубликатов:", bike.duplicated().sum())

print("\nУникальных значений:")
print(bike.nunique())

mismatch = (bike["casual"] + bike["registered"] - bike["cnt"]).abs()
print("\nМаксимальное расхождение casual+registered и cnt:", mismatch.max())
```

```
Пропуски по столбцам:
instant       0
dteday        0
season        0
yr            0
mnth          0
holiday       0
weekday       0
workingday    0
weathersit    0
temp          0
atemp         0
hum           0
windspeed     0
casual        0
registered    0
cnt           0
dtype: int64

Полных дубликатов: 0

Уникальных значений:
instant       731
dteday        731
season          4
yr              2
mnth           12
holiday         2
weekday         7
workingday      2
weathersit      3
temp          499
atemp         690
hum           595
windspeed     650
casual        606
registered    679
cnt           696
dtype: int64

Максимальное расхождение casual+registered и cnt: 0
```

Пропусков и полных дубликатов нет — источник уже агрегирован и подготовлен, удалять нечего. Максимальное расхождение между `casual + registered` и `cnt` равно 0 — значит `cnt` действительно является суммой двух других столбцов, данные внутренне согласованы.

In [24]:
```
for col in ["season", "yr", "mnth", "holiday", "weekday", "workingday", "weather
    print(f"{col:11s} {sorted(bike[col].unique().tolist())}")
```

```
season      [1, 2, 3, 4]
yr          [0, 1]
mnth        [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
holiday     [0, 1]
weekday     [0, 1, 2, 3, 4, 5, 6]
workingday  [0, 1]
weathersit  [1, 2, 3]
```

Значения кодов не выходят за документированные диапазоны (`season` 1–4, `weathersit` 1–4 и т.д.). При этом `weathersit` фактически принимает только значения 1–3 — самая тяжёлая категория погоды (4, сильный дождь/снег) в этом наборе не встретилась ни разу. Это не ошибка, а особенность конкретных двух лет наблюдений.

Оптимизация dtype

`dteday` перевожу в `datetime64` — без этого не построить временную агрегацию. Категориальные коды (`season` , `weathersit` , `weekday` , `holiday` , `workingday` , `yr`) — в `category` , они принимают всего 2–7 повторяющихся значений на 731 строку.

In [25]:
```
mem_before = bike.memory_usage(deep=True).sum()

bike["dteday"] = pd.to_datetime(bike["dteday"])
for col in ["season", "yr", "mnth", "holiday", "weekday", "workingday", "weather
    bike[col] = bike[col].astype("category")

mem_after = bike.memory_usage(deep=True).sum()

print(f"До:    {mem_before / 1024:.2f} KB")
print(f"После: {mem_after / 1024:.2f} KB")
print(f"Изменение: {(1 - mem_after / mem_before) * 100:.1f}%")
display(bike.dtypes)
```

```
До:    98.64 KB
После: 56.77 KB
Изменение: 42.4%
```

```
instant                int64
dteday        datetime64[us]
season              category
yr                  category
mnth                category
holiday             category
weekday             category
workingday          category
weathersit          category
temp                 float64
atemp                float64
hum                  float64
windspeed            float64
casual                 int64
registered             int64
cnt                    int64
dtype: object
```

Память изменилась на 42.4% — на 731 строке эффект `category` небольшой (в отличие от FiresRu с его 26 681 строкой), но `dteday` теперь настоящая дата, с которой можно агрегировать по месяцам, а не текстовая строка. Это самая важная часть этого шага, а не экономия памяти сама по себе.

4. EDA

4.1 Вычисляемые признаки

Два признака разного смысла. `temp_c` / `hum_pct` — обратное преобразование нормировки источника в физические единицы (не новая информация, но иначе цифры 0.34 или 0.80 ничего не говорят). `casual_share_pct` — доля разовых поездок в общем числе за день, содержательный признак для интерпретации поведения пользователей.

In [26]:
```
bike["temp_c"] = (bike["temp"] * 47 - 8).round(1)
bike["hum_pct"] = (bike["hum"] * 100).round(1)
bike["casual_share_pct"] = (bike["casual"] / bike["cnt"] * 100).round(2)

display(bike[["temp", "temp_c", "hum", "hum_pct", "casual", "registered", "cnt",
print("\nСтатистика casual_share_pct:")
display(bike["casual_share_pct"].describe())
```

	temp	temp_c	hum	hum_pct	casual	registered	cnt	casual_share_pct
0	0.344167	8.2	0.805833	80.6	331	654	985	33.60
1	0.363478	9.1	0.696087	69.6	131	670	801	16.35
2	0.196364	1.2	0.437273	43.7	120	1229	1349	8.90
3	0.200000	1.4	0.590435	59.0	108	1454	1562	6.91
4	0.226957	2.7	0.436957	43.7	82	1518	1600	5.12

```
Статистика casual_share_pct:
```

```
count    731.000000
mean      17.559781
std       10.737580
min        1.320000
25%        9.930000
50%       14.880000
75%       21.480000
max       51.030000
Name: casual_share_pct, dtype: float64
```

В среднем на разовые поездки приходится 17.6% от общего числа за день, но разброс большой (от 1.3% до 51.0%) — похоже, доля разовых пользователей заметно зависит от типа дня (будни или выходной), это стоит проверить группировкой ниже.

4.2 Составной фильтр

Отбираю тёплые рабочие дни с ясной погодой — условия, при которых логично ожидать высокий и предсказуемый спрос на аренду (в отличие от выходных, где спрос формируют скорее разовые поездки).

In [27]:
```
warm_workdays = bike.query("workingday == 1 and weathersit == 1 and temp_c > 20"

print("Строк после фильтра:", len(warm_workdays),
      f"({len(warm_workdays) / len(bike) * 100:.1f}% набора)")
print("Средний cnt в этом срезе:", round(warm_workdays["cnt"].mean(), 1))
print("Средний cnt по всему набору:", round(bike["cnt"].mean(), 1))
display(warm_workdays[["dteday", "season", "temp_c", "weathersit", "cnt", "casua
```

```
Строк после фильтра: 127 (17.4% набора)
Средний cnt в этом срезе: 5893.3
Средний cnt по всему набору: 4504.3
```

	dteday	season	temp_c	weathersit	cnt	casual_share_pct
114	2011-04-25	2	20.5	1	4073	18.98
115	2011-04-26	2	21.7	1	4400	15.41
144	2011-05-25	2	23.1	1	4978	14.87
145	2011-05-26	2	25.3	1	4677	16.21
146	2011-05-27	2	24.0	1	4679	18.62

Условию соответствуют 127 дней (17.4% набора). Средний `cnt` в этой подвыборке (5893) заметно выше среднего по всему набору (4504) — тёплая ясная погода в рабочий день действительно связана с более активной арендой, что логично для маршрутов "дом — работа".

4.3 Группировка №1 — сезонность спроса

Средний и медианный `cnt` по сезонам — простое сравнение категорий.

In [28]:
```
season_labels = {1: "Winter", 2: "Spring", 3: "Summer", 4: "Fall"}
bike["season_label"] = bike["season"].astype(int).map(season_labels)

season_cnt = (
    bike.groupby("season_label", observed=True)["cnt"]
        .agg(mean="mean", median="median")
        .round(1)
        .sort_values("mean", ascending=False)
)
display(season_cnt)
```

season_label	mean	median
Summer	5644.3	5353.5
Spring	4992.3	4941.5
Fall	4728.2	4634.5
Winter	2604.1	2209.0

Summer даёт наибольший средний спрос (5644 поездок в день), а Winter — наименьший (2604). Разница в разы говорит о сильной сезонности — велопрокат явно погодозависимый сервис.

4.4 Группировка №2 — рабочие и выходные дни, несколько агрегатов сразу

Для каждого типа дня (`workingday`) сразу считаю размер группы и средние по всем интересующим показателям: общий спрос, разовые/зарегистрированные поездки и долю разовых.

In [29]:
```
workday_stats = (
    bike.groupby("workingday", observed=True)
        .agg(
            n=("cnt", "size"),
            cnt_mean=("cnt", "mean"),
            casual_mean=("casual", "mean"),
            registered_mean=("registered", "mean"),
            casual_share_mean=("casual_share_pct", "mean"),
        )
        .round(1)
)
display(workday_stats)
```

workingday	n	cnt_mean	casual_mean	registered_mean	casual_share_mean
0	231	4330.2	1371.1	2959.0	29.1
1	500	4584.8	606.6	3978.2	12.3

Общее число поездок в рабочие и нерабочие дни отличается не так сильно, а вот структура спроса — сильно: в нерабочие дни доля разовых поездок заметно выше. Это подтверждает гипотезу из раздела 4.1 — велосипед в будни чаще берут по подписке для регулярных поездок, а в выходные — разово, для досуга.

4.5 Статистика, корреляция и потенциальные выбросы

Смотрю на температуру, влажность и общее число поездок: описательную статистику, корреляцию с числом поездок и возможные выбросы по правилу IQR.

In [30]:
```
num_cols = ["temp_c", "hum_pct", "windspeed", "casual", "registered", "cnt"]
display(bike[num_cols].describe().round(1).T)

print("\nКорреляция с cnt:")
print(bike[num_cols].corr()["cnt"].round(3).sort_values(ascending=False))
```

	count	mean	std	min	25%	50%	75%	max
temp_c	731.0	15.3	8.6	-5.2	7.8	15.4	22.8	32.5
hum_pct	731.0	62.8	14.2	0.0	52.0	62.7	73.0	97.2
windspeed	731.0	0.2	0.1	0.0	0.1	0.2	0.2	0.5
casual	731.0	848.2	686.6	2.0	315.5	713.0	1096.0	3410.0
registered	731.0	3656.2	1560.3	20.0	2497.0	3662.0	4776.5	6946.0
cnt	731.0	4504.3	1937.2	22.0	3152.0	4548.0	5956.0	8714.0

```
Корреляция с cnt:
cnt           1.000
registered    0.946
casual        0.673
temp_c        0.628
hum_pct      -0.101
windspeed    -0.235
Name: cnt, dtype: float64
```

В отличие от FiresRu, здесь корреляции заметно сильнее: `registered` и `casual` — очевидно, обе компоненты суммы `cnt` (r ≈ 0.95 и r ≈ 0.67), но и `temp_c` даёт заметную положительную связь (r ≈ 0.63) — чем теплее, тем больше поездок. Это уже не "слабая тенденция", как было с погодой в FiresRu, а достаточно выраженная зависимость.

In [31]:
```
q1 = bike["cnt"].quantile(0.25)
q3 = bike["cnt"].quantile(0.75)
iqr = q3 - q1
lower_bound, upper_bound = q1 - 1.5 * iqr, q3 + 1.5 * iqr
outlier_mask = (bike["cnt"] < lower_bound) | (bike["cnt"] > upper_bound)

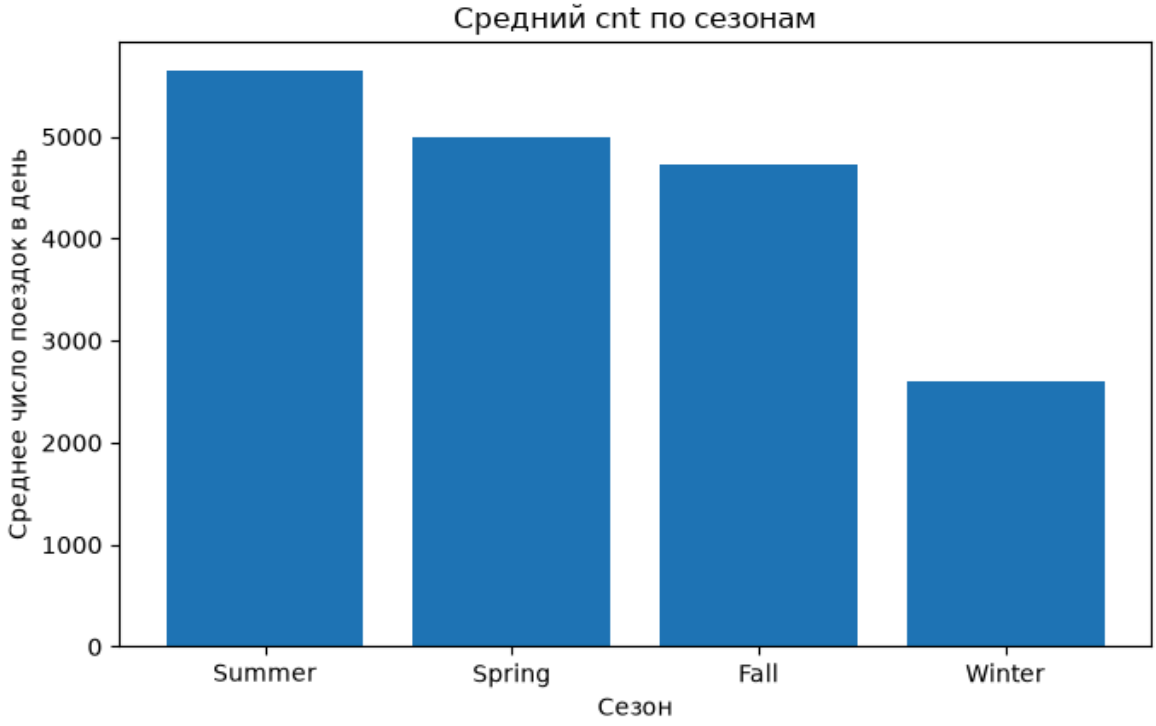
print(f"Границы IQR по cnt: [{lower_bound:.0f}; {upper_bound:.0f}]")
print("Фактический диапазон cnt: [", bike["cnt"].min(), ";", bike["cnt"].max(),
print("Кандидатов на выброс:", outlier_mask.sum())
```

```
Границы IQR по cnt: [-1054; 10162]
Фактический диапазон cnt: [ 22 ; 8714 ]
Кандидатов на выброс: 0
```

Границы IQR получились шире фактического диапазона `cnt` (22–8714) — по этому правилу выбросов нет вообще, ни сверху, ни снизу. В отличие от `wind_speed_ms` в FiresRu (тяжёлый правый хвост, 13,7% кандидатов), распределение суточного спроса на аренду достаточно компактное и без экстремальных дней — само по себе это тоже содержательный результат: не каждый числовой признак обязан иметь выбросы.

4.6 Визуализация №1 — сравнение категорий (сезоны)

In [32]:
```
order = season_cnt.index
plt.figure(figsize=(7, 4.5))
plt.bar(order, bike.groupby("season_label", observed=True)["cnt"].mean().reindex
plt.title("Средний cnt по сезонам")
plt.xlabel("Сезон")
plt.ylabel("Среднее число поездок в день")
plt.tight_layout()
plt.show()
```



4.7 Визуализация №2 — распределение общего спроса (cnt)

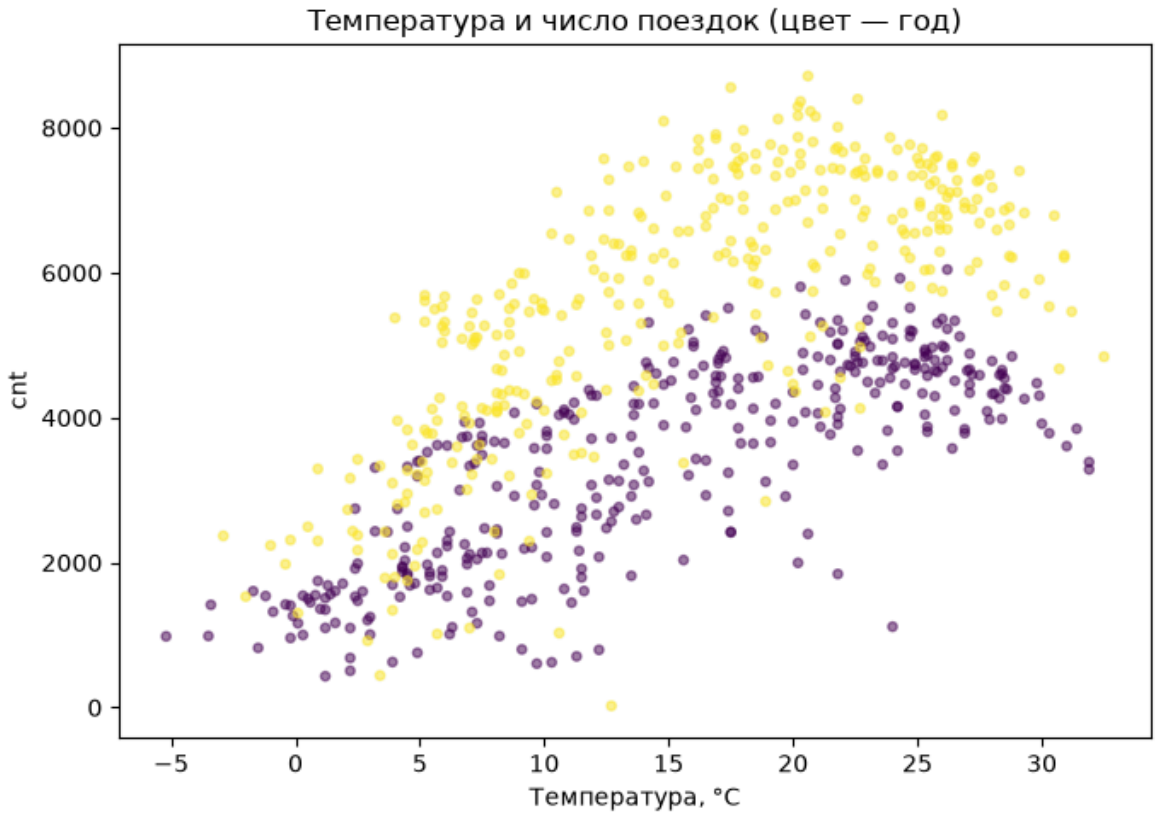
In [33]:
```
plt.figure(figsize=(7, 4.5))
plt.hist(bike["cnt"], bins=30, edgecolor="white")
plt.title("Распределение суточного числа поездок")
plt.xlabel("cnt")
plt.ylabel("Частота")
plt.tight_layout()
plt.show()
```



Распределение похоже на смесь двух режимов — есть заметная группа дней с низким спросом (холодный сезон/плохая погода) и основная масса дней со спросом выше 4000 поездок.

4.8 Визуализация №3 — взаимосвязь температуры и спроса

In [34]:
```
plt.figure(figsize=(7, 5))
plt.scatter(bike["temp_c"], bike["cnt"], s=14, alpha=0.5, c=bike["yr"].astype(in
plt.title("Температура и число поездок (цвет — год)")
plt.xlabel("Температура, °C")
plt.ylabel("cnt")
plt.tight_layout()
plt.show()
```



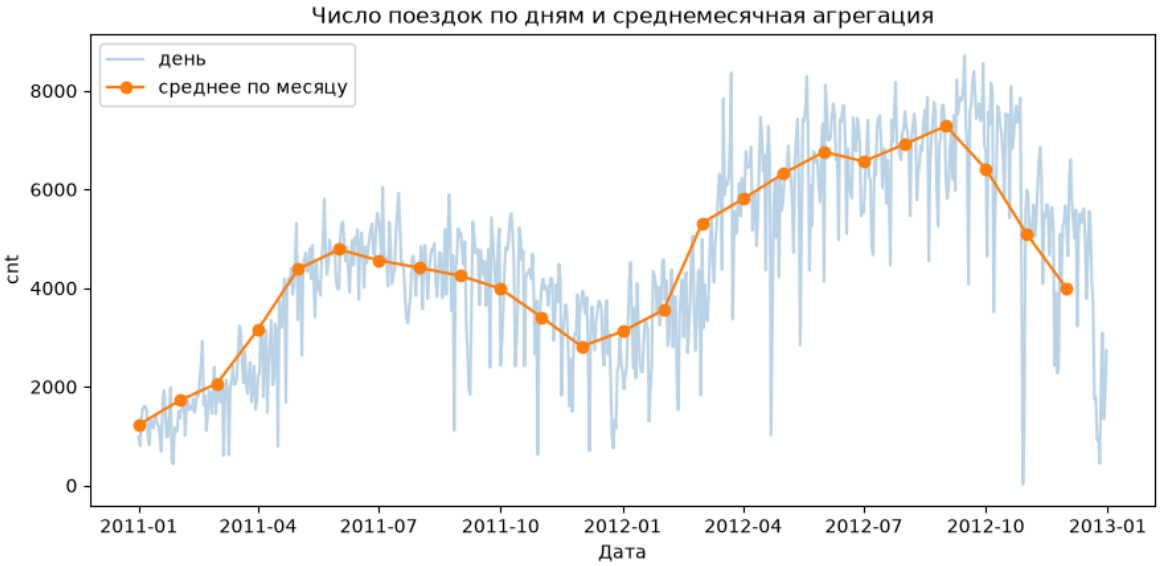
Видна отчётливая положительная связь: с ростом температуры число поездок растёт, хотя при очень высокой температуре рост немного замедляется (жара уже не так комфортна для поездки). Более тёмные точки (2012 год) в среднем выше — сервис явно вырос за второй год наблюдений.

4.9 Визуализация №4 — временной ряд (обязателен, так как есть дата)

Дневной ряд зашумлён, поэтому дополнительно строю агрегацию по месяцам — это и есть требуемая "временная агрегация" для набора с датой.

In [35]:
```
monthly = bike.set_index("dteday")["cnt"].resample("MS").mean()

plt.figure(figsize=(9, 4.5))
plt.plot(bike["dteday"], bike["cnt"], alpha=0.3, label="день")
plt.plot(monthly.index, monthly.values, marker="o", label="среднее по месяцу")
plt.title("Число поездок по дням и среднемесячная агрегация")
plt.xlabel("Дата")
plt.ylabel("cnt")
plt.legend()
plt.tight_layout()
plt.show()
```



Среднемесячная агрегация явно показывает сезонную волну: минимум приходится на зимние месяцы (2011-01, ≈1232 поездок в день), максимум — на тёплый сезон (2012-09, ≈7286). Хорошо видно и общий рост между 2011 и 2012 годом — второй год почти везде выше первого на том же месяце.

5. Экспорт

Сохраняю подготовленную таблицу (с новыми признаками и оптимизированными типами) в CSV, JSON Lines и Parquet, в ту же папку `output`, что и для FiresRu, но под своим именем файлов.

In [36]:
```
from pathlib import Path

OUT_DIR = Path("output")
OUT_DIR.mkdir(exist_ok=True)

bike.to_csv(OUT_DIR / "BikeSharing_prepared.csv", index=False, encoding="utf-8")
bike.to_json(OUT_DIR / "BikeSharing_prepared.json", orient="records", lines=True
             force_ascii=False, date_format="iso")
bike.to_parquet(OUT_DIR / "BikeSharing_prepared.parquet", index=False,
                engine="pyarrow", compression="snappy")

for path in OUT_DIR.glob("BikeSharing_prepared.*"):
    print(f"{path.name:28s} {path.stat().st_size / 1024:7.1f} KB")
```

```
BikeSharing_prepared.csv         71.9 KB
BikeSharing_prepared.json       227.3 KB
BikeSharing_prepared.parquet     61.4 KB
```

In [37]:
```
csv_back = pd.read_csv(OUT_DIR / "BikeSharing_prepared.csv")
parquet_back = pd.read_parquet(OUT_DIR / "BikeSharing_prepared.parquet")

print("CSV:", csv_back.shape, " dtype dteday:", csv_back["dteday"].dtype)
print("Parquet:", parquet_back.shape, " dtype dteday:", parquet_back["dteday"].d
print("\ndtype season после CSV:", csv_back["season"].dtype)
print("dtype season после Parquet:", parquet_back["season"].dtype)
```

```
CSV: (731, 20)  dtype dteday: str
Parquet: (731, 20)  dtype dteday: datetime64[us]

dtype season после CSV: int64
dtype season после Parquet: int64
```

Здесь картина интереснее, чем в части по FiresRu. После CSV, как и ожидалось, `dteday` снова стал обычной строкой, а `season` — числом: текстовый формат не хранит схему вообще. Parquet восстановил `dteday` как настоящий `datetime64` без единой дополнительной строки кода — то есть дата, в отличие от CSV, осталась датой. А вот `season` после Parquet тоже вернулся как `int64`, а не `category` — в отличие от `type_name` в части по FiresRu. Разница в том, что там `category` была построена из **текстовых** значений, а здесь — из **целочисленных** кодов: в этой связке версий pandas/pyarrow категориальный признак на основе чисел через Parquet сохраняется только на уровне значений, а не как отдельный `category`-dtype. Значит рассчитывать, что "Parquet сам всё восстановит", можно не для любой категории — это стоит явно проверять round-trip'ом, а не считать само собой разумеющимся.

6. Итог

Выводы:

1. Набор содержит 731 строку и 16 столбцов, пропусков и полных дубликатов нет; `cnt` во всех строках точно равен сумме `casual` и `registered` — данные внутренне согласованы.
2. Спрос на аренду сильно сезонный: летние месяцы дают в разы больше поездок, чем зимние, и это видно и по группировке, и по временной агрегации.
3. Температура положительно связана с числом поездок (заметно сильнее, чем любая погодная корреляция в FiresRu), но связь не строго линейна — при очень высокой температуре рост спроса замедляется.
4. Структура спроса отличается по типу дня: доля разовых (`casual`) поездок заметно выше в выходные и праздники, чем в рабочие дни — сервис используется по-разному в будни и на отдыхе.
5. Сервис заметно вырос за второй год наблюдений (2012 в среднем выше 2011 по тем же месяцам) — это не сезонный, а трендовый эффект.
6. По правилу IQR у `cnt` нет ни одного выброса ни сверху, ни снизу — распределение суточного спроса компактное, без тяжёлых хвостов, в отличие от `wind_speed_ms` в FiresRu (там кандидатов было 13,7%).

Ограничения анализа:

1. Данные агрегированы по дням для одного города и одной сети велопроката — выводы нельзя переносить на другие города или почасовую динамику без отдельной проверки.
2. Единица измерения `windspeed` в источнике не уточнена (известен только делитель нормировки, 67), поэтому абсолютное значение скорости ветра в

реальных единицах не восстановлено — использовалась только нормированная величина.